

Part I (8 marks)

Part A: Multiple-Choice Questions (each 0.5 mark):

1. Which of the following objects cannot change its content after it is created?

- a) `String obj = new String("CPCS203");`
- b) `StringBuilder obj = new StringBuilder("CPCS203");`
- c) `StringBuilder obj = new StringBuilder("CPCS203");`
- d) `StringBuilder obj = new StringBuilder(203);`

2. Which of the following is a correct difference between the `StringBuilder` and `StringBuffer`?

- a) The `StringBuilder` is immutable and `StringBuffer` is mutable
- b) **The update methods in `StringBuffer` are synchronized**
- c) `StringBuffer` can be used wherever a `String` is used, not like `StringBuilder`
- d) **No difference and they are exactly the same**

3. Which of the following code lines prints “false”?

- a) `System.out.println("CPCS203".matches(".{4}203"));`
- b) `System.out.println("CPCS203".matches("\\D*\\d*"));`
- c) `System.out.println("CPCS203".matches("CP\\.\\d{203}"));`
- d) `System.out.println ("CPCS203".matches("CPCS\\d*") );`

4. Which of the following statements will produce a `TRUE` value?

- a) `"Java" == new String("Java")`
- b) `new String("Java") == new String("Java")`
- c) `"JAVA".equals("Java")`
- d) `new String("Java").equals("Java")`

5. A weather monitoring system stores temperature records in a text file named "temperature.txt". You need to create a `Scanner` object to read data from this file. Which of the following statements correctly initializes the `Scanner` for this task?

- a) `new Scanner("temperature.txt");`
- b) `new Scanner(temperature.txt);`
- c) `new Scanner(File("temperature.txt"));`
- d) `new Scanner(new File("temperature.txt"));`

6. Consider the following Java program, which defines a Calculator class with a method that adds two numbers. The class hides its internal data while providing a simple interface for users:

```
class Calculator {
    private int result; // Internal data hidden from direct access
    public void add(int a, int b) {
        result = a + b;
        System.out.println("Result: " + result);
    }
}

public class Test {
    public static void main(String[] args) {
        Calculator calc = new Calculator();
        calc.add(5, 3);
    }
}
```

Which object-oriented principle is demonstrated in the above code, where the implementation details of the `Calculator` class are hidden while providing a simple interface for users?

- a) Encapsulation
- b) Aggregation
- c) Inheritance
- d) Abstraction

7. If you want this object to be not referenced by any variable so JVM can automatically collect as garbage, you can explicitly assign the value.

- a) false
- b) garbage
- c) '\oooo'
- d) null

Part B: Output-Based Questions

1. What is the output of the following program?

[1.5 mark]

```
import java.lang.StringBuffer;
public class TestMain (
    public static void main(String[] args) {
        String str = "Thank YOU! you R Good!";
        System.out.println(str.substring(0, 6) +
            str.substring(11, 14));
        System.out.println(str.charAt(8) + str.substring(14, 22));
        StringBuilder sb = new StringBuilder(str.substring(16,21));
        System.out.println(sb.insert(0, "Very")));
    }
}
```

Output:

2. What is the output of the following program

[1.5 mark]

```
public class ChangeText {
    public static void main(String[] args) {
        String str = "Java";
        StringBuilder builder = new StringBuilder(str);
        System.out.println(str + " and " + builder);

        change(str, builder);

        System.out.println(str);
        System.out.println(builder);
    }

    private static void change(String str2, StringBuilder builder2) {
        str2 = str2 + " OOP!";
        builder2.append(" OOP!");
    }
}
```

Output:

3. What is the output of the following program?

[1.5 mark]

```
public class TestSplit {  
    public static void main(String[] args) {  
  
        String str = "Java#Python#C##C++#";  
  
        String[] group1 = str.split("#", 0);  
        System.out.println(group1.length);  
  
        String[] group2 = str.split("#", -1);  
        System.out.println(group2.length);  
  
        String[] group3 = str.split("#", 3);  
        System.out.println(group3[2]);  
    }  
}
```

Output:

Part II (8 marks)

Answer the following questions:

Q1. A Java program is reading data from this file. What will be the output of the following code?

[1.0 Mark]

A text file (Scores.txt) contains the following data:

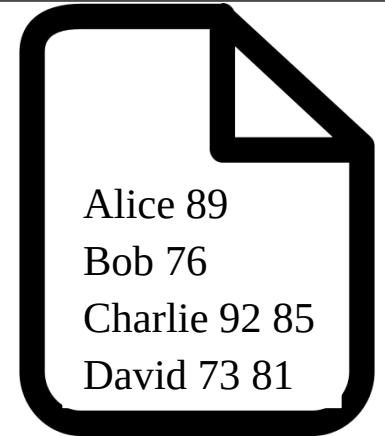
```
File F1 = new File("Scores.txt");
Scanner in = new Scanner(F1);

System.out.println("Student: " + in.next());
System.out.println("Score1: " + in.nextInt());

if (in.next().equals("Charlie")) {
    System.out.println("High Score: " + in.nextInt());
} else {
    System.out.println("Low Score: " + in.nextInt());
}

in.nextLine();
System.out.println("Final Score: " + in.nextLine());

if (in != null)
    in.close();
```



The output:

Q2. What will be the output when the following testing class is executed? Assume that all classes (**Person, Employee, Manager, and the testing class**) are in the same package. [2.5 Mark]

<pre>public class Person { private int id; private String name; Person(int id, String name) { this.id = id; this.name = name; System.out.println(id + ":" + name); } public void display(double x, int y) { System.out.println(x + y); } }</pre>	<pre>public class Employee extends Person { String employeeID; Employee() { super(10, "None"); this.employeeID ="BOT00";} Employee(int id, String name,String employeeID) { super(id, name); this.employeeID = employeeID; } Public void display(int y, double x) { System.out.println(x - y); } }</pre>
<pre>public class Manager extends Employee { String department; Manager(String department) { this.department = department; System.out.println(department); } Manager() { this("None"); System.out.println("Manager"); } @Override Public void display(double x, int y) { System.out.println(x * y); } }</pre>	<pre>public class Testing { public static void main(String[] args) { Manager mgr = new Manager(); mgr.display(3, 4.0); mgr.display(4.0, 4); } }</pre>

Output:

Q3. What will be the output of the following program?

[1 Mark]

```
class BankAccount {
    private double balance;

    public BankAccount(double initialBalance) {
        if (initialBalance > 0)
            balance = initialBalance;
    }

    public void deposit(double amount) {
        if (amount > 0)
            balance += amount;
    }

    public void withdraw(double amount) {
        if (amount > 0 && amount <= balance)
            balance -= amount;
    }

    public double getBalance() {
        return balance;
    }
}

public class BankingApp {
    public static void main(String[] args) {
        BankAccount acc1 = new BankAccount(1000);
        acc1.deposit(500);
        acc1.withdraw(1200);
        System.out.println("Account 1 Balance: " + acc1.getBalance());

        BankAccount acc4 = new BankAccount(0);
        acc4.withdraw(10);
        acc4.deposit(100);
        acc4.withdraw(90);
        System.out.println("Account 4 Balance: " + acc4.getBalance());
    }
}
```

The Output:

Account 1 Balance:

Account 4 Balance:

Q4. Moral and Emad are keeping track of their daily expenses in a file. The program first creates the file if it doesn't exist and writes initial expenses. If the file already exists, it reads and updates the expenses by adding 10 to each value and saves the new expenses in a new file. [2 Marks]

What will be the output of the following program when executed twice?

```
import java.io.*;
import java.util.Scanner;

public class ExpenseTracker {
    public static void main(String[] args) throws IOException {
        File file = new File("expenses.txt");

        if (!file.exists()) {
            PrintWriter writer = new PrintWriter(file);
            writer.println("Morad 200");
            writer.println("Emad 150");
            writer.close();
        } else {
            Scanner scanner = new Scanner(file);
            PrintWriter writer = new PrintWriter("updated_expenses.txt");

            while (scanner.hasNext()) {
                String name = scanner.next();
                int expense = scanner.nextInt();
                writer.println(name + " " + (expense+10)); //Increase expense by 10
            }
            scanner.close();
            writer.close();
        }

        // Read and print the updated file (if it exists)
        file = new File("updated_expenses.txt");
        if (file.exists()) {
            Scanner readFile = new Scanner(file);
            while (readFile.hasNextLine()) {
                System.out.println(readFile.nextLine());
            }
            readFile.close();
        }
    }
}
```

Expected Output(First Run)	Expected Output(Second Run)

Q.5 Below is an example of a mutable Java class named Student. Please rewrite it so that it becomes immutable. When you handle the class to be immutable, make sure that we can still retrieve the id, name, age, and the marks. [1.5 marks]

Note: You do **not** need to rewrite the entire Student class. Instead, **only modify the necessary parts** to make the class **immutable**.

```
public class Student
{
    private int id;
    private String name;
    private int age;
    private int[] marks;

    public Student(int id, String name, int age, int[] marks) {
        this.id = id;
        this.name = name;
        this.age = age;
        this.marks = marks;
    }

    public String getName() {
        return this.name;
    }

    public int[] getMarks() {
        return this.marks;
    }

    public int getAge() {
        return this.age;
    }
}
```

Part III (9 marks)

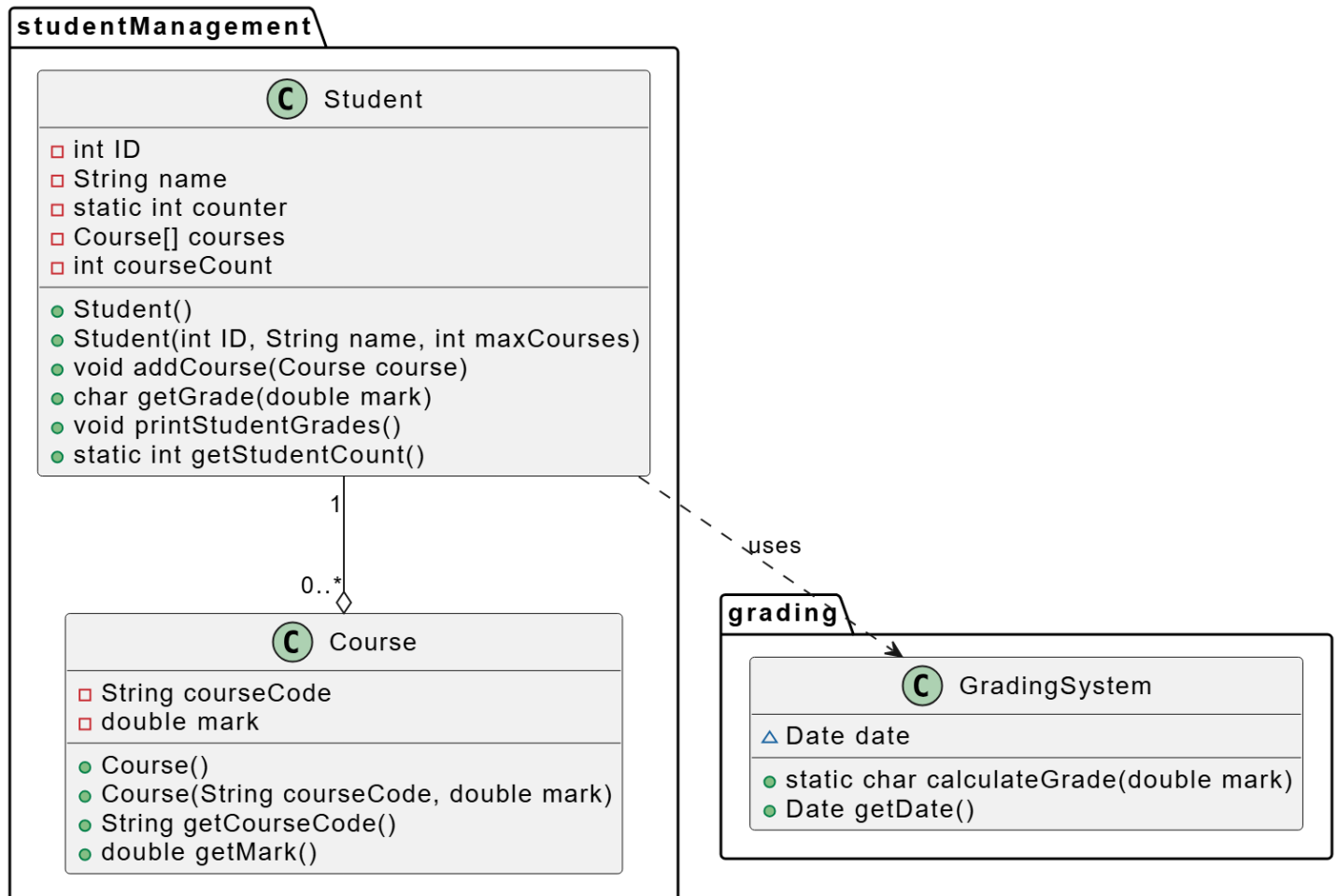
Answer the following Question:

Refer to the following UML class diagram to answer **Questions 1 to 6**.

The diagram represents **three classes**:

- The Student and Course classes belong to the **Student Management package**.
- The GradingSystem class belongs to a **different package**, named **Grading**.

Note: Consider this UML diagram when answering all six questions.



Note: indicates **default access**.

indicates **public access**.

indicates **private access**.

Q1. Match each pair of classes with the correct relationship type:

[1.5 Marks]

Note: Write the correct answer in the blank spaces provided.

Class 1	Class 2	Relationship Type (Aggregation, Dependency, No Relationship)
Student	Course	
Student	GradingSystem	
Course	GradingSystem	

Q2. Assume that in the Student class, the programmer has created an object of GradingSystem and wants to access the date field. However, a syntax error occurs. Explain why this error happened.

[1 Mark]

Q3. Consider the following Java code:

```
public class Main {  
    public static void main(String[] args) {  
        Student student1 = new Student(101, "Ahmed", 3);  
        Student student2 = new Student(102, "Khalid", 2);  
        Student student3 = new Student(); // Default constructor  
  
        System.out.println("Total Students: " + Student.getStudentCount());  
        System.out.println("Student 3 ID: " + student3.getID());  
        System.out.println("Student 3 Name: " + student3.getName());  
    }  
}
```

What will be the output of this program? [1.5 Marks]

Total Students:

Student 3 ID:

Student 3 Name:

Q.4 Write the appropriate Java constructor for the Student class that initializes the ID, name, and number of courses, as specified in the UML diagram.

Your constructor should:

1. Initialize the ID and name using the provided parameters. [0.5 Marks]
2. Allocate memory for the courses array based on the given number of courses . [0.5 Marks]
3. Increment the static counter for each Student object created. [0.5 Marks]

Q5. Consider the following Java statement:

```
Student[] students = new Student[5];
```

How many **Student objects** are created by the above statement? [1 Mark]

- A. 0
- B. 5
- C. 4
- D. None of the above

Q6. Consider the following Java code that initializes an array of Student objects, assigns them courses, and iterates over the array to print their grades:

```
Student[] students = new Student[3];
students[0] = new Student(101, "Ahmed", 3);
students[1] = new Student(102, "Ali", 3);
students[2] = new Student(103, "Khalid", 3);
students[0].addCourse(new Course("CPCS 203", 95));
students[1].addCourse(new Course("CPCS 203", 85));
students[2].addCourse(new Course("CPCS 203", 50));

// Loop to iterate over students and print their grades
for (Student student : students) {
    student.printStudentGrades();
}
```

Task:

Complete the **printStudentGrades** method inside the Student class. This method should:

- Print the student's **name and ID**. [0.5 Mark]
- Iterate through the student's **courses** and print each course's name, mark, and grade. [1.5 Marks]
- Use the **calculateGrade()** method from the GradingSystem class to determine the student's grade, **as calculateGrade() is assumed to be fully implemented.** [0.5 Mark]

```
public void printStudentGrades() {
```

```
}
```

Expected Output After Implementation:

Student: Ahmed (ID: 101)
Course: CPCs 203 | Mark: 95 | Grade: A

Student: Ali (ID: 102)
Course: CPCs 203 | Mark: 85 | Grade: B

Student: Khalid (ID: 103)
Course: CPCs 203 | Mark: 50 | Grade: F